

Tarea S4.01. BigQuery Avanzado & Analytics Engineering

Contenido

Nivel 1: Entorno e Ingesta Híbrida (Code-First)	1
Ejercicio 1: Consulta sobre Tabla no Optimizada (Diagnóstico)	1
Ejercicio 2: Re-arquitectura y Optimización del Almacenamiento (Partition & Cluster) ..	1
Ejercicio 3: La Prueba del Algodón (Benchmark)	4
Ejercicio 4: Smart Caching (Vistas Materializadas).....	7
Nivel 2: SQL Analítico Avanzado	9
Ejercicio 1: Perfilado de Clientes VIP (Métricas Agregadas con CTEs)	9
Ejercicio 2: Análisis de Tendencias (Window Functions sobre Vistas)	10
Ejercicio 3: Totales Acumulados (Running Totales sobre Vistas).....	11
Ejercicio 4: Fidelización y Valor del Cliente (Filtraje Avanzado)	12
Nivel 3: Analytics Engineering (Arrays & Automatización).....	14
Ejercicio 1: Desanimiento y Allanamiento de Datos (Unnesting)	14
Ejercicio 2: El Ranking de Ventas (Agregación Simple).....	16
Ejercicio 3: Automatización del Pipeline y Visualización.....	17

Nivel 1: Entorno e Ingesta Híbrida (Code-First)

Ejercicio 1: Consulta sobre Tabla no Optimizada (Diagnóstico)

El Country Manager de Alemania necesita revisar urgentemente las transacciones del día 12 de marzo de 2022 .

🔗 Tarea:

- » 1. Escribe la consulta que une (JOIN) transacciones y compañías.
- » 2. Filtra los resultados por la fecha indicada y el país "Germany".
- » 3. Sin ejecutar la consulta, realiza un "Dry Run" (auditoría de costes).

🔗 Observación: Fíjate en que BigQuery lee casi toda la mesa a pesar de pedir sólo un día (Full Table Scan).

Ejecución del Dry Run

```
1 SELECT t.amount, t.declined, DATE(t.timestamp) AS date
2 FROM `sprint3-analytics-alexregueira.sprint3_silver.transactions_clean` AS t
3 JOIN `sprint3-analytics-alexregueira.sprint3_silver.companies_clean` AS co
4 ON t.business_id = co.company_id
5 WHERE co.country = 'Germany' AND DATE(t.timestamp) = '2022-03-12'
6 ORDER BY t.amount DESC;
```

✅ This query will process 3.05 MB when run.

Resultados de la consulta:

Row	amount	declined	date
1	589.15	0	2022-03-12
2	589.15	0	2022-03-12
3	489.29	0	2022-03-12
4	467.0	0	2022-03-12
5	188.58	0	2022-03-12

Ejercicio 2: Re-arquitectura y Optimización del Almacenamiento (Partition & Cluster)

Escenario:

Tras el diagnóstico del ejercicio anterior, hemos confirmado que filtrar la tabla de transacciones por fecha o por empresa provoca un Full Table Scan (lectura completa de la

tabla), lo que es ineficiente. Para solucionarlo en la capa de consumo (Gold), materializaremos una versión optimizada de la tabla de hechos.

Problema del sandbox: al utilizar un entorno gratuito sin tarjeta de crédito, BigQuery borra automáticamente los datos de las particiones con más de 60 días de antigüedad. Dado que nuestros datos históricos son antiguos, si particionamos directamente la tabla, ésta acabará quedando vacía. Para evitarlo, lo resolveremos en dos pasos: primero, actualizaremos automáticamente las fechas de los datos y después aplicaremos la optimización física.

Tarea:

Paso 1: Generación de Datos Recientes (Mocking Data)

Crea una tabla intermedia llamada `sprint3_silver.transactions_recent` a partir de la tabla `sprint3_silver.transactions_clean`. Tu objetivo es mantener todas las columnas, pero sustituir el timestamp original por uno nuevo, generado aleatoriamente para que caiga dentro de los últimos 50 días.

» Pista Técnica: Para hacerlo sin escribir todas las columnas a mano, puedes utilizar `SELECT * EXCEPT(timestamp)`. Para calcular la fecha aleatoria, deberás restar una cantidad de días al instante actual. Te serán muy útiles las funciones `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, y la combinación `CAST(RAND() * 50 AS INT64)` para generarlos días de desfase.

Paso 2: Creación de la Mesa Optimizada (Partitioning & Clustering)

Ahora, crea tu tabla definitiva `sprint3_gold.fact_transactions_optimized` a partir de los datos recientes que acabas de generar en `transactions_recent`. Tienes que construir la sentencia DDL para configurar la tabla con las siguientes estrategias físicas de almacenamiento:

» Particionamiento (Partitioning): Divide la tabla por la fecha del campo `DATE(timestamp)`. Esto permitirá que las consultas que filtren por día (`WHERE date = ...`) sólo lean la partición necesaria.

» Clustering: Ordena los datos dentro de cada partición por `business_id`. Esto acelerará drásticamente los filtros por compañía y los cruces (JOINS) con la dimensión de empresas.

Entrega:

» El código SQL utilizado para el Paso 1.

» El código DDL (`CREATE OR REPLACE TABLE...`) utilizado en el Paso 2.

» Una captura de pantalla de los "Detalles" de la nueva tabla `fact_transactions_optimized` donde se pueda comprobar que está particionada y agrupada en clústeres correctamente.

Creación de la tabla transactions_recent:

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-alexregueira.sprint3_silver.transactions_recent` AS
2 SELECT *
3 EXCEPT(timestamp),
4 | TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY) AS timestamp
5 FROM `sprint3-analytics-alexregueira.sprint3_silver.transactions_clean`;
```

✓ Query completed

Using on-demand processing quota

> Query results

Job information **Results** Execution details Execution graph

ⓘ This statement replaced the table named transactions_recent.

Creación de la tabla fact_transactions_optimized:

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-alexregueira.sprint3_gold.fact_transactions_optimized`
2 PARTITION BY DATE(timestamp)
3 CLUSTER BY business_id
4 AS
5 SELECT *
6 FROM `sprint3-analytics-alexregueira.sprint3_silver.transactions_recent`;
```

✓ Query completed

Using on-demand processing quota

> Query results

Job information **Results** Execution details Execution graph

ⓘ This statement created a new table named fact_transactions_optimized.

Información del particionamiento:

Table info	
Table ID	sprint3-analytics-alexregueira.sprint3_gold.fact_transactions_optimized
Created	Jul 29, 2026, 11:09:15 UTC+2
Last modified	Jul 29, 2026, 11:09:15 UTC+2
Table expiration	NEVER
Data location	EU
Default collation	
Default rounding mode	ROUNDING_MODE_UNSPECIFIED
Case insensitive	false
Description	
Labels	
Primary key(s)	
Tags	
Table Type	Partitioned
Partitioned by	DAY
Partitioned on field	timestamp
Partition expiration	60 days
Partition filter	Not required
Clustered by	business_id

Ejercicio 3: La Prueba del Algodón (Benchmark)

Escenario:

Escenario : Ya has creado la mesa = `sprint3_gold.fact_transactions_optimized` con las estrategias de Particionamiento y Clustering. Ahora toca demostrar al equipo la mejora de rendimiento y el ahorro de costes que has conseguido. Simularemos una petición típica de negocio: "Queremos ver todas las transacciones de los últimos 30 días".

🔗 Tarea:

El objetivo es claro y directo: aplicar exactamente la misma consulta a dos tablas diferentes y comparar su coste computacional .

Construye una consulta SQL que seleccione todas las columnas (`SELECT *`) y filtre los datos de los últimos 30 días.

⚠️ NO ejecutes las consultas todavía. En este ejercicio sólo utilizaremos el validador de BigQuery ("Dry Run", arriba a la derecha del editor) para visualizar los Bytes procesados de forma gratuita.

Haz la comparativa siguiendo estos dos pasos:

» Paso 1 (Tabla no optimizada): Apunta tu consulta en la tabla `sprint3_silver.transactions_recent`. Observa el validador y realiza una captura de pantalla de los bytes que procesaría.

» Paso 2 (Tabla optimizada): Sin cambiar nada de la lógica de tu consulta, modifica sólo el FROM porque ahora apunte a la tabla particionada sprint3_gold.fact_transactions_optimized. Observa cómo cambia el validador y realiza la segunda captura.

📁 Entrega:

- » El código SQL base que has diseñado.
- » Las dos capturas de pantalla donde se vea la diferencia de "Bytes procesados" en el validador.
- » Un breve texto calculando cuál es el % de ahorro que has conseguido (compara los bytes del Paso 1 respecto a los del Paso 2).

Consulta en la tabla no optimizada:

```
1 SELECT *
2   EXCEPT(timestamp),
3   DATE(timestamp) AS timestamp
4 FROM
5   `sprint3-analytics-alexregueira.sprint3_silver.transactions_recent`
6 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
7 ORDER BY timestamp DESC;
```

Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details	Execution graph					
Row	transaction_id	card_id	business_id	amount	declined	product_ids	user_id	lat	longitude	timestamp
1	3F2720A9-C1C9...	CcU-4856	b-2390	189.72	0	81	1	40.713271730220953	-74.00448266281...	2026-07-29
						94				
						65				
2	77A28844-2597...	CcU-4849	b-2242	267.6	0	15	2	39.956313953554549	-75.17202022528...	2026-07-29
						40				
3	7B4E4335-2F8E...	CcU-4842	b-2294	650.59	0	92	3	29.770344231679594	-95.379571201844	2026-07-29
						41				
						23				
						3				
4	5BD1FD50-BFE4...	CcU-4828	b-2286	184.52	0	75	5	39.950112050135111	-75.16386608532...	2026-07-29
						84				
						47				
5	52236CE7-8DC4...	CcU-4814	b-2590	254.09	0	15	7	41.87847946398761	-87.62674130171...	2026-07-29
						43				

Detalles del coste de procesamiento:

1

2

3

4

5

6

7

```
SELECT *
  EXCEPT(timestamp),
  DATE(timestamp) AS timestamp
FROM
  `sprint3-analytics-alexregueira.sprint3_silver.transactions_recent`
WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
ORDER BY timestamp DESC;
```

Query completed

Using on-demand processing quota

>|

Query results

Job information	Results	Visualization	JSON	Execution details
Duration	1 sec 49 ms			
Bytes processed	11.85 MB			
Bytes billed	12 MB			
Slot milliseconds	776			

Consulta en la tabla optimizada:

1

2

3

4

5

6

7

```
SELECT *
  EXCEPT(timestamp),
  DATE(timestamp) AS timestamp
FROM
  `sprint3-analytics-alexregueira.sprint3_gold.fact_transactions_optimized`
WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
ORDER BY timestamp DESC;
```

Query completed

Using on-demand processing quota

>|

Query results

Job information	Results	Visualization	JSON	Execution details	Execution graph					
Row	transaction_id	card_id	business_id	amount	declined	product_ids	user_id	lat	longitude	timestamp
1	84C3A5F0-25BA-4...	CcS-5327	b-2618	416.81	0	23	746	60.5317831484633	18.77755742426...	2026-07-29
						55				
						4				
						3				
2	19FFC925-CA23-4...	CcS-8651	b-2618	144.96	0	96	4070	51.306732970290447	10.35926472512...	2026-07-29
						50				
3	7B941C0B-CA8E-4...	CcS-7998	b-2618	576.62	0	16	3417	38.710254306843233	-9.146926949119...	2026-07-29
						39				
						17				
						35				
4	0E9D9F3F-92AC-4...	CcS-5842	b-2618	203.78	0	49	1261	59.838828408291576	18.46716480199...	2026-07-29
						100				
						21				
						54				
5	243424EF-9B6E-44...	CcS-6412	b-2618	344.31	0	22	1831	51.584936777459212	10.14359185904...	2026-07-29
						15				

Detalles del coste de procesamiento:

```
1 SELECT *
2   EXCEPT(timestamp),
3   DATE(timestamp) AS timestamp
4 FROM
5   `sprint3-analytics-alexregueira.sprint3_gold.fact_transactions_optimized`
6 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
7 ORDER BY timestamp DESC;
```

✓ Query completed

Using on-demand processing quota

> Query results

Job information	Results	Visualization	JSON	Execution details	Export
Duration	1 sec 54 ms				
Bytes processed	7.21 MB				
Bytes billed	10 MB				
Slot milliseconds	2560				

El hecho de que la tabla estuviera particionada por fecha y luego agrupada por ID de empresa redujo el coste de procesamiento en aproximadamente un 39 % para los bytes procesados y en aproximadamente un 17 % para los bytes facturados.

Ejercicio 4: Smart Caching (Vistas Materializadas)

Escenario: El Director General tiene un cuadro de mando que muestra las " Ventas Totales por Día ". Él (y otros 50 managers) refrescan este gráfico constantemente.

Problema: Si utilizas una consulta normal o una Vista estándar, cada vez que alguien refresca la página, BigQuery recalcula la suma de millones de filas. Es lento y estás pagando por el mismo cálculo una y otra vez.

Solución: Utilizar una Vista Materializada . Esta tecnología guarda el resultado del cálculo en la memoria caché y sólo la actualiza si entran nuevos datos (incremental).

🔗 Tarea: Crea una Vista Materializada llamada `sprint3_gold.mv_daily_sales` que muestre las Ventas Totales por Día .

👉 Entrega:

» Código para Crear la Vista Materializada .

» Haz una consulta que muestre la vista creada y realiza una captura de pantalla que muestre en su conjunto la vista materializada creada, con la consulta ejecutada y los bits procesados.

Creación de la vista materializada:

1
2
3
4
5

```
CREATE OR REPLACE MATERIALIZED VIEW `sprint3-analytics-alexregueira.sprint3_gold.mv_daily_sales` AS
SELECT DATE(timestamp) AS date,
SUM(amount) AS total_sales
FROM `sprint3-analytics-alexregueira.sprint3_gold.fact_transactions_optimized`
GROUP BY date;
```

✓

This query will process 0 B when run.

>|

Query results

Job information

Results

Execution details

Execution graph

i

This statement created a new materialized view named mv_daily_sales.

Consulta ejecutada del total de ventas por día:

1
2
3
4
5
6
7

```
SELECT
date,
ROUND(total_sales, 2) AS total_sales
FROM `sprint3-analytics-alexregueira.sprint3_gold.mv_daily_sales`
ORDER BY date DESC
LIMIT 10;
```

✓

Query completed

Using on-demand processing quota

Processing location: EU

✕

>|

Query results

Job information

Results

Visualization

JSON

Execution details

Row	date	total_sales
1	2026-07-29	261567.21
2	2026-07-28	513797.89
3	2026-07-27	519216.15
4	2026-07-26	511192.06
5	2026-07-25	495904.23
6	2026-07-24	506128.87
7	2026-07-23	515984.5
8	2026-07-22	521942.25
9	2026-07-21	506564.75
10	2026-07-20	510346.26

Detalle de los bytes procesados:

```
1 SELECT
2   date,
3   ROUND(total_sales, 2) AS total_sales
4 FROM `sprint3-analytics-alexregueira.sprint3_gold.mv_daily_sales`
5 ORDER BY date DESC
6 LIMIT 10;
7
```

✓ Query completed

Processing location: EU X

> Query results

Job information	Results	Visualization	JSON	Execution details
Bytes processed	0 B (results cached)			

Se puede observar que cuando la consulta se realiza más de una vez, el valor de los bytes procesados se reduce a cero porque la vista materializada se almacena en la caché, evitando así los cargos por consultas repetidas.

Nivel 2: SQL Analítico Avanzado

Escenario:

La dirección comienza a hacer preguntas que no se resuelven con un simple GROUP BY. Necesitamos análisis de tendencias y comportamiento de usuario.

Ejercicio 1: Perfilado de Clientes VIP (Métricas Agregadas con CTEs)

Escenario: El equipo de Marketing quiere analizar el comportamiento de nuestros mejores clientes para diseñar la estrategia del próximo año. Definimos "VIP" como aquellos con un gasto acumulado superior a 500 €. Necesitan un informe que, para cada VIP, muestre su nombre, contacto y su patrón de compra: cuántas veces ha comprado, cuánto gasta de media y cuál fue su compra récord.

🔗 Tarea:

» 1. Crea una CTE llamada VIP_Stats que agrupe por usuario y calcule :

El Gasto Total (SUM).

La Cantidad de Transacciones (COUNT).

El Ticket Medio (AVG), redondeado a 2 decimales.

La Compra Máxima (MAX).

Filtro: Mantiene sólo aquellos cuyo Gasto Total sea > 500.

» Crea la CTE con users_combined para obtener los datos personales.

Requisitos de Salida:

Columnas: user_id, nombre_completo, email, num_compras, ticket_medio, max_compra, total_gastado.

Ordenado por total_gastado descendente.

Consulta del perfil de compras por usuario:

```
1 WITH VIP_Stats AS (  
2   SELECT  
3     user_id,  
4     CONCAT(name, ' ', surname) AS nombre_completo,  
5     email  
6   FROM `sprint3-analytics-alexrequeira.sprint3_silver.users_combined` AS u  
7 )  
8  
9 SELECT  
10  v.user_id,  
11  v.nombre_completo,  
12  v.email,  
13  COUNT(transaction_id) AS num_compras,  
14  ROUND(AVG(amount),2) AS ticket_medio,  
15  ROUND(MAX(amount),2) AS max_compra,  
16  ROUND(SUM(amount),2) AS total_gastado  
17 FROM `sprint3-analytics-alexrequeira.sprint3_silver.transactions_clean` AS t  
18 JOIN VIP_Stats AS v  
19 ON t.user_id = v.user_id  
20 WHERE t.declined = 0  
21 GROUP BY  
22  v.user_id,  
23  v.nombre_completo,  
24  v.email  
25 HAVING total_gastado > 500  
26 ORDER BY total_gastado DESC;
```

✓ This query will process 6.17 MB when run.
Using on-demand processing quota Processing location: EU ✕

Row	user_id	nombre_completo	email	num_compras	ticket_medio	max_compra	total_gastado
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwlll Phylgnmhnd	fwlll.phylgnmhnd@example.com	66	522.6	724.44	34491.27
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.co.uk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgyfridxs	sfzzoh.xgyfridxs@example.com	78	304.0	604.9	23711.9

Results per page: 50 1 - 50 of 5000

Ejercicio 2: Análisis de Tendencias (Window Functions sobre Vistas)

Escenario: La Dirección Financiera quiere monitorear la "velocidad de ventas" diaria. Necesitan un informe que compare el rendimiento de cada día contra el día anterior para detectar caídas bruscas o picos de crecimiento (Day-over-Day Growth).

 Estrategia de Optimización: Para este cálculo NO debes ir a la tabla de hechos original ni recalculiar sumas. Como buen Data Engineer, debes utilizar la Vista Materializada sprint3_gold.mv_daily_sales que creaste en el ejercicio anterior. Esto garantiza que el análisis de tendencias sea instantáneo y no consuma recursos de cómputo innecesarios.

 Tarea: Crea una consulta sobre la vista materializada que genere un informe con las siguientes 4 columnas:

» 1. Fecha : La fecha de venta.

- » 2. Ventas_Hoy : El total vendido aquel día.
- » 3. Ventas_Ayer : El total vendido el día anterior (usando funciones de ventana).
- » 4. Diff_Percentual : El porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeado a 2 decimales.

💡 Pista Técnica: Utiliza la función LAG(camp) OVER (ORDER BY data) para leer el valor de la fila anterior sin necesidad de realizar JOINS complejos.

Resultado de la consulta utilizando una tabla CTE para realizar la captura del valore del día anterior:

The screenshot shows a SQL query editor with a query that uses a Common Table Expression (CTE) to calculate daily sales and their differences. The query is as follows:

```

1 WITH Calculo_ventas AS (
2   SELECT
3     date AS Fecha,
4     total_sales AS Ventas_Hoy,
5     LAG(total_sales) OVER (ORDER BY date) AS Ventas_Ayer
6   FROM `sprint3-analytics-alexrequeira.sprint3_gold.mv_daily_sales`
7 )
8
9 SELECT
10  Fecha,
11  ROUND(Ventas_Hoy, 2) AS Ventas_Hoy,
12  ROUND(Ventas_Ayer, 2) AS Ventas_vs_Ayer,
13  ROUND((Ventas_Hoy - Ventas_Ayer) / Ventas_Ayer * 100, 2) AS Diff_Percentual
14 FROM Calculo_ventas
15 ORDER BY Fecha DESC;

```

Below the query, the results are displayed in a table with the following columns: Row, Fecha, Ventas_Hoy, Ventas_vs_Ayer, and Diff_Percentual. The results show data for the period from 2026-07-22 to 2026-07-29.

Row	Fecha	Ventas_Hoy	Ventas_vs_Ayer	Diff_Percentual
1	2026-07-29	261567.21	513797.89	-49.09
2	2026-07-28	513797.89	519216.15	-1.04
3	2026-07-27	519216.15	511192.06	1.57
4	2026-07-26	511192.06	495904.23	3.08
5	2026-07-25	495904.23	506128.87	-2.02
6	2026-07-24	506128.87	515984.5	-1.91
7	2026-07-23	515984.5	521942.25	-1.14
8	2026-07-22	521942.25	506564.75	3.04

Ejercicio 3: Totales Acumulados (Running Totales sobre Vistas)

Escenario: El Director Financiero (CFO) necesita visualizar la curva de crecimiento anual. No le interesa tanto lo que se vendió hoy, sino cuánto llevamos facturado en total desde el inicio del año hasta la fecha (YTD - Year To Date).

🔧 Tarea: Utilizando la vista materializada mv_daily_sales (para no recalculer agregaciones base), genera un informe con tres columnas:

- » 1. Fecha.
- » 2. Ventas del Día: Redondeado a 2 decimales.
- » 3. Ventas Acumuladas YTD: Una suma progresiva de las ventas que debe reiniciarse cada 1 de Enero . El resultado final debe estar redondeado a 2 decimales.

💡 Pista Técnica: Para conseguir el reinicio anual, necesitas dividir la ventana usando PARTITION BY EXTRACT(YEAR FROM data). La estructura completa sería: SUM(...) OVER

(PARTITION BY... ORDER BY... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Resultado de la consulta utilizando tabla CTE para realizar la suma de YTD:

```
1 WITH Calculo_ventas AS (  
2   SELECT  
3     date AS Fecha,  
4     total_sales AS Ventas_Hoy,  
5     SUM(total_sales) OVER (  
6       PARTITION BY EXTRACT(YEAR FROM date)  
7       ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
8     ) AS Ventas_acumuladas  
9   FROM `sprint3-analytics-alexregueira.sprint3_gold.mv_daily_sales`  
10 )  
11  
12 SELECT  
13   Fecha,  
14   ROUND(Ventas_Hoy, 2) AS Ventas_Hoy,  
15   ROUND(Ventas_acumuladas, 2) AS Ventas_acumuladas  
16 FROM Calculo_Ventas  
17 ORDER BY Fecha DESC  
18 LIMIT 5;
```

✓ Query completed

> Query results

Job information	Results	Visualization	JSON	Execution det
Row	Fecha	Ventas_Hoy	Ventas_acumula...	
1	2026-07-29	261567.21	25901531.17	
2	2026-07-28	513797.89	25639963.96	
3	2026-07-27	519216.15	25126166.07	
4	2026-07-26	511192.06	24606949.92	
5	2026-07-25	495904.23	24095757.86	

Ejercicio 4: Fidelización y Valor del Cliente (Filtraje Avanzado)

Escenario: El equipo de Marketing lanza la campaña "A la tercera va la vencida". Quieren enviar un regalo a los usuarios que hayan completado su tercera transacción. Sin embargo, el presupuesto del regalo no es fijo: depende de la "calidad" del cliente en sus inicios. Necesitan saber el Ticket Medio de estas 3 primeras compras para segmentar si envían un detalle básico o un regalo premium.

🔗 Tarea: Genera un listado de los usuarios que han llegado (o superado) su tercera compra, mostrando:

- » 1. Datos de usuario (user_id, nombre_completo, email) .
- » 2. La fecha y el importe exacto de la 3ª compra .
- » 3. Media de las 3 primeras : La media de gasto de sus transacciones 1, 2 y 3.

💡 Pista Técnica: Debes utilizar la función ROW_NUMBER() para establecer el orden cronológico de las compras. Tienes que utilizar la cláusula QUALIFY para filtrar. Reto de

Optimización : Busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el historial histórico es costoso. Trate de filtrar las filas necesarias antes de realizar la agregación final.

Para crear una lista personalizada con las últimas tres compras de cada cliente, se puede utilizar una secuencia de CTE (Elementos de Transacción del Cliente). El proceso comienza con los datos personales, seguido de la búsqueda de las últimas tres compras de cada usuario y la fecha de cada una, y finalmente, el cálculo del promedio de estas tres transacciones. Por último, todos los resultados de los CTE se combinan en una consulta que muestra los datos personales, la fecha y el valor de la tercera compra, y el promedio de las últimas tres. A partir de ahí, se puede establecer un valor mínimo para clasificar la calidad de las compras de los clientes y, de esta forma, ofrecerles un regalo adecuado.

```
1 WITH Datos_usuarios AS(
2   SELECT
3     user_id,
4     CONCAT(name, ' ', surname) AS nombre_completo,
5     email
6   FROM `sprint3-analytics-alexregueira.sprint3_silver.users_combined` AS u
7 ),
8
9 Primeras_3_Compras AS (
10  SELECT
11    user_id,
12    DATE(timestamp) AS fecha,
13    amount,
14    ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp ASC) AS num_compra
15  FROM `sprint3-analytics-alexregueira.sprint3_gold.fact_transactions_optimized`
16  QUALIFY num_compra <= 3
17 ),
18
19 Calculo_Final AS (
20  SELECT
21    user_id,
22    fecha AS fecha_3ra_compra,
23    amount AS importe_3ra_compra,
24    AVG(amount) OVER (PARTITION BY user_id) AS media_3_primeras,
25    num_compra
26  FROM Primeras_3_Compras
27  QUALIFY num_compra = 3
28 )
29
30 SELECT
31   u.user_id,
32   u.nombre_completo,
33   u.email,
34   c.fecha_3ra_compra,
35   ROUND(c.importe_3ra_compra, 2) AS importe_3ra_compra,
36   ROUND(c.media_3_primeras, 2) AS media_3_primeras
37 FROM Datos_usuarios AS u
38 JOIN Calculo_Final AS c
39   ON u.user_id = c.user_id
40 ORDER BY u.user_id;
```

Query results							Chat	View all jobs
Job information		Results		Visualization	JSON	Execution details	Execution graph	
Row	user_id	nombre_completo	email	fecha_3ra_compra	importe_3ra_compra	media_3_primeras		
1	1	Zeus Gamble	interdum.enim@protonmail.edu	2026-06-18	212.74	289.98		
2	2	Garrett McConnell	integer.vitae.nibh@protonmail.org	2026-06-20	529.36	428.24		
3	3	Ciaran Harrison	interdum.feugiat@aol.org	2026-06-11	556.96	367.39		
4	4	Howard Stafford	omare.egestas@icloud.edu	2026-06-17	282.29	247.4		
5	5	Hayfa Pierce	et.malesuada.fames@hotmail.org	2026-06-11	167.15	265.89		

Nivel 3: Analytics Engineering (Arrays & Automatización)

Escenario:

Marketing necesita urgentemente un Ranking de Productos más Vendidos.

Ejercicio 1: Desanimiento y Allanamiento de Datos (Unnesting)

Escenario: Actualmente, en la tabla de transacciones, los productos vendidos están "escondidos" dentro de un Array (una lista de IDs). El equipo de ventas quiere analizar qué productos específicos se venden más, pero no pueden hacerlo si los datos están comprimidos en una lista. Necesitan una Tabla Detallada (Flat Table) donde cada fila represente un producto vendido, aunque esto signifique que el ID de la transacción se repita.

 Tarea: Crea la mesa dim_transactions_flat desnormalizando la información. Tienes que "explotar" el array de productos y cruzarlo con el catálogo maestro para obtener los nombres y precios individuales.

 Pasos Técnicos:

» 1. Utiliza CROSS JOIN UNNEST(product_ids) para transformar el Array en filas individuales.

» 2. Realiza uno JOIN con la tabla products para obtener el nombre (name) y precio (price) de cada ítem.

📍 Nota: Ten cuidado con los tipos de datos al hacer el JOIN (Array de INT64 vs Product_ID STRING).

» 3. No agrupe el resultado. Queremos ver el desglose línea por línea.

☒ Resultado Esperado (Visualización):

Tu mesa final debe verse "plana", donde una transacción con 3 productos ocupa 3 filas:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portátil Básico	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratón	12.99

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclado	45.50

🔗 Entrega: Una captura de la Vista Previa de tu tabla en la que se vea el DDL ejecutada correctamente. Una captura de pantalla donde se vea el esquema de la nueva tabla.

Para crear la tabla `gold.dim_transactions_flat`, se utilizó una cadena **WITH**. La primera función establece un código SKU para cada nombre de producto utilizando la función **CASE WHEN ROWNUMBER() OVER (ORDER BY name)**, utilizando valores ASCII para su sufijo. La segunda función divide la columna ID de producto de la tabla `transactions_clean` utilizando la función **CROSS JOIN UNNEST(product_ids) AS id_producto_extraido**, completando las filas con información de la transacción a la que pertenece el producto y con información derivada de la tabla `products_clean`. Finalmente, la información requerida se une en la tabla `gold.dim_transactions_flat`, y el tipo de datos de la columna `product_id` en la tabla `products_clean` se transforma a INT64 para que se pueda utilizar un LEFT JOIN con la columna `id_producto_extraido` de la función **WITH Transacciones_Explodidas**.

Creación de la tabla `gold.dim_transactions_flat`:

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-alexregueira.sprint3_gold.dim_transactions_flat` AS
2
3 WITH Productos_Numerados AS (
4   SELECT
5     name,
6     CONCAT(
7       'SKU_',
8       CASE
9         WHEN ROW_NUMBER() OVER (ORDER BY name) <= 26 THEN CHR(64 + ROW_NUMBER() OVER (ORDER BY name))
10        ELSE CONCAT(CHR(64 + DIV(ROW_NUMBER() OVER (ORDER BY name) - 1, 26)), CHR(65 + MOD(ROW_NUMBER() OVER (ORDER BY name) - 1, 26)))
11      END
12    ) AS product_sku
13   FROM (
14     SELECT DISTINCT name
15     FROM `sprint3-analytics-alexregueira.sprint3_silver.products_clean`
16     WHERE name IS NOT NULL
17   )
18 ),
19 Transacciones_Explodidas AS (
20   SELECT
21     SELECT
```

✓ This query will process 7.09 MB when run.

Using on-demand processing quota Processing location: EU ✕

Query results Chat View all jobs Save results Open in

Job information Results Execution details Execution graph

ⓘ This statement replaced the table named dim_transactions_flat. Go to table

Resultados de la consulta:

Row	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price
1	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2024-08-28 07:16:46 UTC	395.43	SKU_BN	the duel warden	180.91
2	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2024-08-28 07:16:46 UTC	395.43	SKU_T	Stark Karstark	53.01
3	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2024-08-28 07:16:46 UTC	395.43	SKU_AP	jinn Winterfell	65.25
4	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2024-08-28 07:16:46 UTC	395.43	SKU_BE	sith Jade	96.26
5	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2016-12-21 20:07:18 UTC	155.63	SKU_AR	mustafar jinn	2.12
6	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2016-12-21 20:07:18 UTC	155.63	SKU_P	Littlefinger the giantsblood	57.25
7	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2016-12-21 20:07:18 UTC	155.63	SKU_BE	sith Jade	96.26
8	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2020-07-14 15:37:45 UTC	326.01	SKU_K	Karstark warden	79.53
9	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2020-07-14 15:37:45 UTC	326.01	SKU_J	Karstark Dorne	49.7
10	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2020-07-14 15:37:45 UTC	326.01	SKU_BN	the duel warden	180.91

Schema creado:

☐	Field name	Type	Mode
☐	transaction_id	STRING	NULLABLE
☐	timestamp	TIMESTAMP	NULLABLE
☐	total_ticket	FLOAT	NULLABLE
☐	product_sku	STRING	NULLABLE
☐	product_name	STRING	NULLABLE
☐	product_price	FLOAT	NULLABLE

Ejercicio 2: El Ranking de Ventas (Agregación Simple)

Escenario: Ahora que has preparado la infraestructura y dispones de la mesa `dim_transactions_flat` (donde cada fila equivale a un producto vendido), generar informes analíticos es trivial. Ya no necesitas funciones complejas de desapego para responder a preguntas básicas de negocio.

🔗 Tarea: Genera el Top 5 de productos más vendidos en la historia de la compañía.

🔗 Método: Consulta directamente tu nueva tabla desnormalizada (`dim_transactions_flat`). Al ser la tabla plana, simplemente necesitas una agrupación estándar (`GROUP BY`) para el nombre del producto y una cuenta (`COUNT`) de las filas.

🔗 Entrega: El código SQL que genera el ranking. Una captura del resultado con los 5 productos ganadores y cuántas unidades se han vendido de cada uno.

Consulta simplificada de los 5 productos más vendidos:

```
1 SELECT COUNT(product_name) AS unidades_vendidas, product_name
2 FROM `sprint3-analytics-alexregueira.sprint3_gold.dim_transactions_flat`
3 GROUP BY product_name
4 ORDER BY unidades_vendidas DESC
5 LIMIT 5;
```

✓ Query completed

Using on-demand processing quota Processing location: EU ✕

>| Query results

Job information	Results	Visualization	JSON	Execution details
Row	unidades_vendidas	product_name		
1	10071	Winterfell		
2	10059	the duel		
3	10022	dooku solo		
4	7694	Karstark Dorne		
5	7607	skywalker ewok		

Ejercicio 3: Automatización del Pipeline y Visualización

Escenario: El informe estático del ejercicio anterior ha gustado mucho, pero el Director de Marketing necesita profesionalizarlo para la toma de decisiones diaria. Tiene dos requerimientos urgentes:

1. Lógica Centralizada: Quieren ver el precio con IVA (21%) desglosado. Sin embargo, dado que el impuesto puede cambiar por país o año, te prohíben poner el cálculo $\text{precio} * 1,21$ directamente en el código ("Hardcoded"). Tienes que utilizar una función reutilizable.

2. Frescor de Datos: El tablero debe estar actualizado automáticamente cada mañana a las 07:00 AM, antes de la reunión diaria de ventas.

🔗 Tarea: Tienes que evolucionar tu tabla `dim_transactions_flat` para incluir el cálculo de impuestos y automatizar su regeneración.

⚙️ Pasos Técnicos:

1. User Defined Functions (UDF): Crea una función SQL persistente llamada `calculate_tax(amount)` que reciba un valor numérico y devuelva el resultado aplicando el 21% de impuesto.

2. Integración y Orquestación: Modifica el código de creación de la tabla (del Ejercicio 1) para que utilice tu nueva función `calculate_tax` y genere una nueva columna: `product_price_tax_inc`. Configura una Scheduled Query en BigQuery para que esta sentencia (CTAS) se ejecute automáticamente todos los días a las 07:00 AM.

3. Visualización (BI): Conecta Looker Studio a tu mesa `dim_transactions_flat` y crea un Dashboard: "Monitor de Rendimiento de Ventas"

📎 Entrega: El código SQL de la UDF; El código SQL actualizado de la creación de la tabla; Una captura de pantalla demostrando que la Scheduled Query está programada; Un vínculo del gráfico a Looker Studio.

La función persistente **`calculate_tax`** genera un cálculo sumando el tipo de IVA al precio del producto, de modo que cuando se llama en **`SELECT calculate_tax(product_price)`** calcula el precio de cada producto con el impuesto aplicado.

Nivel 3 - 3

Run

Save

Schedule

Share

FileEditRunTools

```

1
2 CREATE TEMP FUNCTION calculate_tax(precio FLOAT64)
3 RETURNS FLOAT64 AS (
4   precio * (1 + 0.21)
5 );
6
7 SELECT
8   transaction_id,
9   product_name,
10  product_price AS precio_sin_iva,
11  ROUND(calculate_tax(product_price), 2) AS precio_con_iva
12
13 FROM `sprint3-analytics-alexregueira.sprint3_gold.dim_transactions_flat`
14 ORDER BY transaction_id;

```

Query completed

Using on-demand processing quota

Processing location: EU

>|

Query results

Job information	Results	Visualization	JSON	Execution details	Execution gra
Row	transaction_id	product_name	precio_sin_iva	precio_con_iva	
1	00043A49-2949-494B-A5DD-A5...	the duel warden	180.91	218.9	
2	00043A49-2949-494B-A5DD-A5...	Stark Karstark	53.01	64.14	
3	00043A49-2949-494B-A5DD-A5...	jinn Winterfell	65.25	78.95	
4	00043A49-2949-494B-A5DD-A5...	sith Jade	96.26	116.47	
5	000447FE-B650-4DCF-85DE-C7...	mustafar jinn	2.12	2.57	

Adaptando el código del ejercicio anterior que genera y actualiza la tabla **dim_transactions_flat** para agregar el cálculo de impuestos para los precios de los productos, comenzamos definiendo la función que establece el valor del IVA que se aplicará, el cual, cuando sea necesario, solo necesita actualizarse en una sola línea de código. Luego, se utiliza el código para generar nuestra tabla. Para implementar la nueva columna con el impuesto aplicado al precio del producto, usamos la llamada a la función **ROUND(calculate_tax(p.price), 2) AS precio_con_iva**, que toma como argumento el precio **p.price** extraído de la tabla **products_raw**.

Nivel 3 - 1

Run

Save

Schedule

Share

File

Edit

Run

Tools

```

1 -- Nueva función temp para actualizar el valor del IVA.
2 CREATE TEMP FUNCTION calculate_tax(precio FLOAT64)
3 RETURNS FLOAT64 AS (
4   precio * (1 + 0.21)
5 );
6
7 -- Creación de la tabla dim_transactions_flat.
8 CREATE OR REPLACE TABLE `sprint3-analytics-alexregueira.sprint3_gold.dim_transactions_flat` AS
9
10 WITH Productos_Numerados AS (
11   SELECT
12     name,
13     CONCAT(
14       'SKU_',
15       CASE
16         WHEN ROW_NUMBER() OVER (ORDER BY name) <= 26 THEN CHR(64 + ROW_NUMBER() OVER (ORDER BY name))
17         ELSE CONCAT(CHR(64 + DIV(ROW_NUMBER() OVER (ORDER BY name) - 1, 26)), CHR(65 + MOD(ROW_NUMBER() OVER (ORDER BY name) - 1, 26)))
18       END
19     ) AS product_sku
20   FROM (
21     SELECT DISTINCT name
22     FROM `sprint3-analytics-alexregueira.sprint3_silver.products_clean`
23     WHERE name IS NOT NULL
24   )
25 ),
26
27 Transacciones_Explodidas AS (
28   SELECT
29     transaction_id,
30     timestamp,
31     amount AS total_ticket,
32     id_producto_extraido
33   FROM `sprint3-analytics-alexregueira.sprint3_silver.transactions_clean`
34   CROSS JOIN UNNEST(product_ids) AS id_producto_extraido
35 )
36
37 SELECT
38   t.transaction_id,
39   t.timestamp,
40   t.total_ticket,
41   sku.product_sku,
42   p.name AS product_name,
43   p.price AS product_price,
44   ROUND(calculate_tax(p.price), 2) AS precio_con_iva -- Nueva columna para el precio con IVA
45
46 FROM Transacciones_Explodidas AS t
47 LEFT JOIN `sprint3-analytics-alexregueira.sprint3_silver.products_clean` AS p
48   ON t.id_producto_extraido = CAST(p.product_id AS INT64)
49 LEFT JOIN Productos_Numerados AS sku
50   ON p.name = sku.name
51 ORDER BY t.transaction_id;

```

Row	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price	precio_con_iva
1	00043A49-2949-494B-ASDD-AS...	2024-08-28 07:16:46 UTC	395.43	SKU_BN	the duel warden	180.91	218.9
2	00043A49-2949-494B-ASDD-AS...	2024-08-28 07:16:46 UTC	395.43	SKU_T	Stark Karstark	53.01	64.14
3	00043A49-2949-494B-ASDD-AS...	2024-08-28 07:16:46 UTC	395.43	SKU_AP	jinn Winterfell	65.25	78.95
4	00043A49-2949-494B-ASDD-AS...	2024-08-28 07:16:46 UTC	395.43	SKU_BE	sith Jade	96.26	116.47
5	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	SKU_AR	mustafar jinn	2.12	2.57
6	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	SKU_P	Littlefinger the giantsblood	57.25	69.27
7	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	SKU_BE	sith Jade	96.26	116.47
8	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	SKU_K	Karstark warden	79.53	96.23
9	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	SKU_J	Karstark Dome	49.7	60.14
10	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	SKU_BN	the duel warden	180.91	218.9

Para programar la ejecución de la consulta, debe acceder al asesor de programación, especificando el proyecto y la tabla donde se almacenará el resultado. A continuación, se establecen la frecuencia y el tiempo de ejecución.

Export as scheduled query

✔ Settings valid.

Details and schedule

Name for scheduled query *

Tax daily update

Schedule options

Repeat frequency *

Days

At *

07:00

UTC



Start now



Start at set time

Start date and run time

9/7/26, 11:08

CET



End never



Schedule end time

End date

CET



Scheduled to run every day at 07:00 UTC starting now.

Destination for query results



Set a destination table for query results

Dataset *

✔ sprint3-analytics-alexregueira.sprint3_gold

Table Id *

dim_transactions_flat

Destination table partitioning field

Destination table write preference



Append to table



Overwrite table



Automatic location selection

Finalmente, podemos visualizar los datos obtenidos mediante la herramienta Data Studio (antes Looker Studio), donde los datos se pueden agrupar y mostrar en gráficos y tablas. De esta forma, para crear un monitor de rendimiento de ventas, podemos agrupar los productos más vendidos en dos gráficos: el primero destacando los 10 productos más vendidos y el segundo mostrando una visión general de las ventas. También podemos incluir el total de ventas por transacción en una tabla, así, al seleccionar una transacción en la tabla, podemos ver en los gráficos la cantidad de productos por transacción y el valor de cada producto vendido.

Enlace para acceder al Dashboard:
<https://datastudio.google.com/u/2/reporting/202038c4-bad5-49d5-8ec3-0b15032198e0/page/tEnnC>

